ARŞİV: ŞG-2026/AR-GE/MMO-0041/R4

HİZMETE ÖZEL / TİCARİ SIR

ŞİMŞEK OYUN VE FİNANS TEKNOLOJİLERİ A.Ş.

YAZILIM MÜHENDİSLİĞİ VE BİLGİ TEKNOLOJİLERİ DİREKTÖRLÜĞÜ

OYUN MANTIĞI VE VERİ ENTEGRASYONU ÇALIŞMA GRUBU

EVRAK KODU: ŞG-2026/AR-GE/MMO-0041/R4
KONU: Business Bank Online — 10 Zustand Store Dönüşümü, 50 İş Kuralının Sunucu Otoritesine Nakli ve Geçiş Planı
TARİH: 19 Ekim 2026
GİZLİLİK: TİCARİ GİZLİ

🔄 RAPOR 4: Mevcut Oyun Analizi & Uyum (Migrasyon) Raporu

Proje: Business Bank Online — MMO Ekonomi Simülasyonu
Hazırlayan: Antigravity Architect
Tarih: 19 Ekim 2026
Revizyon: v1.0

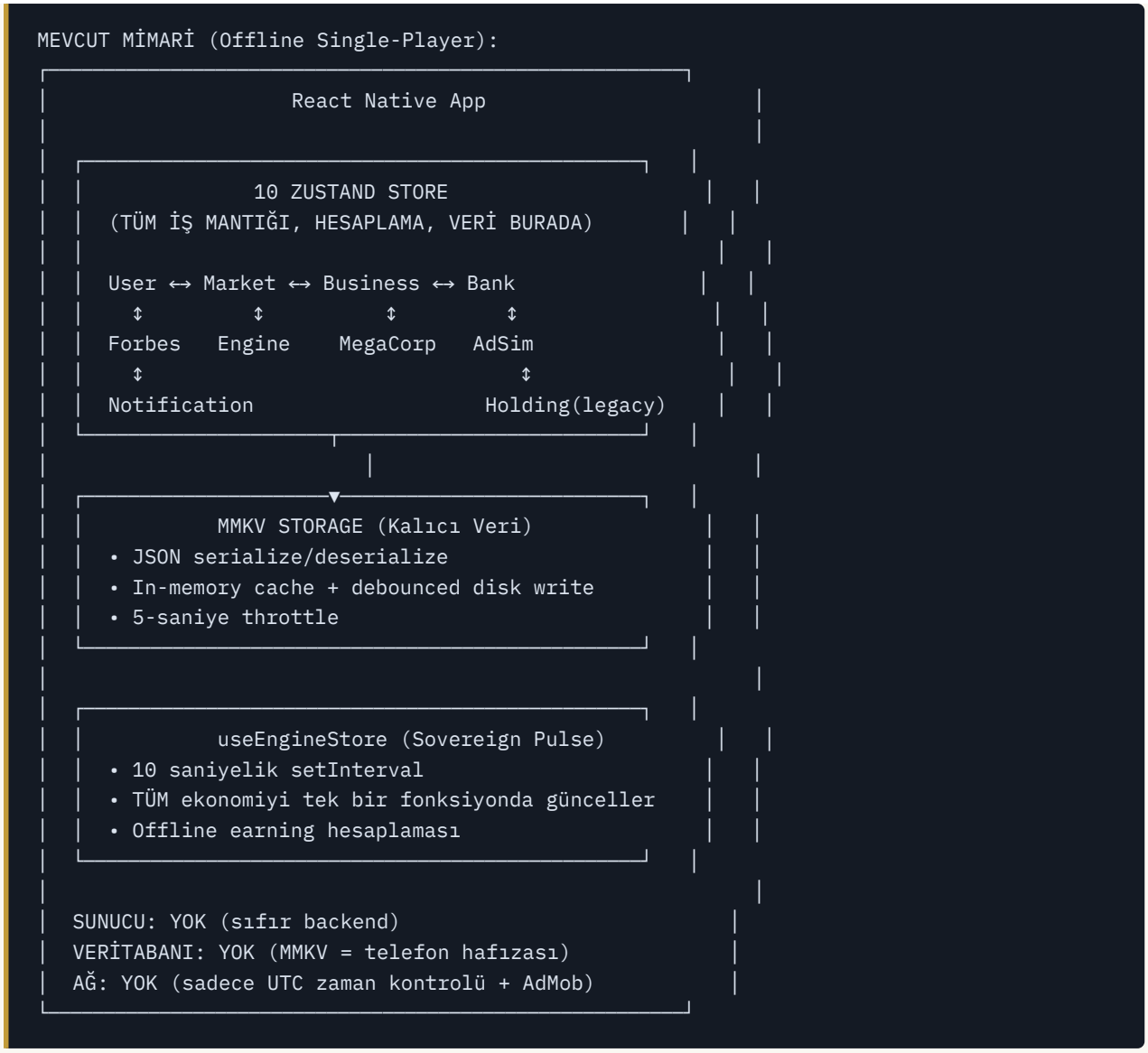
1. Yönetici Özeti

Bu rapor, Business Bank'ın mevcut offline tek-oyuncu yapısını **satır satır** analiz ederek her bileşenin yeni online mimariye **nasıl dönüşeceğini** haritalamaktadır.

Kapsam:
- 10 Zustand store'un tam analizi ve online eşleştirmesi
- 11 ekranın dönüşüm planı
- 20+ bileşenin migrasyon stratejisi
- 150+ iş kuralının sunucu/client dağılımı
- Riskler ve dikkat noktaları
- Faz faz uygulama planı

2. Mevcut Mimari Röntgeni

2.1 GENEL YAPI



2.2 MEVCUT DOSYA ENVANTERİ

KATEGORİ	DOSYA SAYISI	TOPLAM SATIR (TAHMİNİ)
Zustand Stores	10	~4,500
Screens	11	~5,000
Components	20+	~4,000
Data Files	4	~300
Navigation	1	~200
Utils / Services	3	~300
Config / Theme	3	~100
i18n / Locales	5	~5,000
TOPLAM	**~57**	**~19,400**

---

3. Store-by-Store Migrasyon Haritası

3.1 USEUSERSTORE.JS → WALLET SERVİCE + AUTH SERVİCE

Mevcut: 1 devasa store, ~600 satır, 40+ state alanı, 30+ method.

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`balance`, `reservedBalance`	**Server:** **`WalletService`**	DB'de tutulur, client sadece okur
`addBalance(amount)`	**Server:** **`WalletService.credit()`**	Client çağıramaz, sadece sunucu
`handleManualClick()`	**Server:** **`ClickService.processTap()`**	Anti-cheat + rate limit sunucuda
`clickPower`, `clickPowerLevel`, `cardLevel`, `maxCardLevel`, `totalClicks`	**Server:** **`WalletService`** (DB)	Hesaplama sunucuda
`username`, `email`, `profileImage`, `userTitle`, `language`	**Server:** **`AuthService`** / **`UserService`**	Profil bilgileri
`taxStats`, `taxLastPaid`, vergi methods	**Server:** **`TaxService`**	16 saat kilidi sunucuda enforced
`stats` (totalClickEarned, totalBizEarned, vb.)	**Server:** **`StatsService`**	İstatistikler DB'de
`history` (transaction log)	**Server:** **`TransactionService`**	`transactions` tablosu
`overclockEndTime`, `businessTurboEndTime`, vb. booster timer'lar	**Server:** **`BoosterService`**	Zamanlayıcılar sunucuda
`isVIP`, `isPremiumUser`, `isAdsRemoved`	**Server:** **`UserService`**	IAP doğrulama sonrası set
`redeemCode()`	**Server:** **`PromoService`**	Kod doğrulama sunucuda
`formatMoney()`	**Client:** **`@bb/core/utils`**	Saf formatlama, client'ta kalır
`checkDailyVIPReward()`	**Server:** **`RewardService`**	UTC tarih kontrolü sunucuda
`checkMilestones()`	**Server:** **`ForbesService`**	Sıralama sunucuda
`onboardingComplete`, `hasSeenTutorial`	**Client store**	Yerel UI durumu
`selectedCardDesign`, `hasNewCardUnlocked`	**Server** (kalıcı) + **Client** (UI)	
`clickTimestamps` (auto-clicker detection)	**Server:** **`ClickService`**	Rate limiting

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`timeCheatCount`, `lastSyncTime`	**Kaldırılır**	Anti-cheat sunucu tarafında
`usedPromoCodes`	**Server:** `PromoService`	DB'de
`_scheduleSave()`, `rehydrate()`	**Kaldırılır**	MMKV persistence gereksiz

Client'ta Kalan (useWalletStore):

```
// Sadece sunucudan gelen veriyi cache'leyen thin store
interface WalletClientState {
  balance: number;           // Server'dan WS ile güncellenir
  reservedBalance: number;
  cardLevel: number;
  maxCardLevel: number;
  selectedCardDesign: number;
  clickPower: number;
  totalClicks: number;
  taxDebts: TaxDebtSummary;
  stats: PlayerStats;
  boosters: ActiveBoosters;
  // UI-only
  hasNewCardUnlocked: boolean;
}
```

3.2 USEMARKETSTORE.JS → STOCK EXCHANGE SERVİCE + CRYPTO SERVİCE

Mevcut: ~800 satır, 75 hisse + 20 kripto + limit emir sistemi + piyasa simülasyonu.

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`stocks` (75 hisse verisi + fiyat)	**Server:** **StockService** (DB + Redis cache)	Hisseler DB'de, fiyatlar Redis'te
`crypto` (20 token + fiyat)	**Server:** **CryptoService** (DB + Redis)	Aynı
`portfolio`	**Server:** **PortfolioService** (DB)	`stock_holdings` tablosu
`limitOrders`	**Server:** **OrderBookService** (DB)	`limit_orders` tablosu
`tickMarket()`	**Server:** **PriceOracleService**	Sovereign Pulse içinde
Fiyat hesaplama (drift, mean reversion, noise)	**Server:** **PriceCalculatorService**	Tüm formüller sunucuda
`buyStock()`, `sellStock()`	**Server:** **TradeService**	Atomik TX + FOR UPDATE
`addLimitOrder()`, `cancelLimitOrder()`	**Server:** **OrderBookService**	DB persistence
Hostile takeover check (%99 hisse)	**Server:** **TakeoverService**	MegaCorp domain event
`marketCycle`, `marketTrend`	**Server:** **EconomyEngine**	Global ekonomi state
`activeEvent` (TECH_CRASH, vb.)	**Server:** **MarketEventService**	Rastgele event enjeksiyonu
Flash pump/crash (kripto)	**Server:** **CryptoService**	0.3% olasılık
Komisyon hesaplama (0.1% / 0.002% VIP)	**Server:** **TradeService**	VIP durumu sunucudan

Client'ta Kalan (useMarketStore):

```
// Fiyat stream receiver — sadece okuma
interface MarketClientState {
  stocks: StockSummary[];     // WS'den gelen delta updates
  crypto: CryptoSummary[];
  portfolio: PortfolioItem[]; // REST'ten başlangıçta, WS'den güncelleme
  limitOrders: LimitOrder[];
  marketCycle: 'BULL' | 'BEAR' | 'NEUTRAL';
  activeEvent: MarketEvent | null;
}
```

3.3 USEBUSİNESSSTORE.JS → FLEET SERVİCE + REAL ESTATE SERVİCE

Mevcut: ~700 satır, iş yönetimi + gayrimenkul + taksi filosu + lojistik.

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`ownedBusinesses`	**Server:** **`BusinessService`** (DB)	`businesses` tablosu
`realEstate`	**Server:** **`PropertyService`** (DB)	`properties` tablosu
`realEstateMarket` (20 şablon)	**Server** (seed data) + **Client** (constants)	`@bb/core/constants/property-catalog`
`businessTemplates` (8 iş tipi)	**Server** (seed) + **Client** (constants)	`@bb/core/constants/business-templates`
`buyNewBusiness()`	**Server:** **`BusinessService.create()`**	Bakiye kontrolü + kapasite kontrolü sunucuda
`buyProperty()`	**Server:** **`PropertyService.purchase()`**	Stok limiti (5/şablon) + enflasyon hesabı sunucuda
`buyVehicle()`	**Server:** **`VehicleService.purchase()`**	Garaj kapasitesi + bakiye sunucuda
`maintenanceVehicle()`	**Server:** **`VehicleService.repair()`**	Maliyet hesabı sunucuda
`tickBusiness()`	**Server: Sovereign Pulse**	Araç aşınma ve gelir hesabı
`processRealEstateGrowth()`	**Server: Sovereign Pulse**	Günlük %1 değer artışı
`purchaseCounts` (enflasyon)	**Server:** **`PropertyService`**	Global alım sayısı (tüm oyuncular)
`businessCapacity`	**Server:** **`BusinessService`**	Kapasite DB'de
`businessConstruction`	**Server:** **`ConstructionService`**	Timer sunucuda
Tasfiye (%80 refund)	**Server**	Hesaplama sunucuda

> **[BİLGİ / IMPORTANT]:** **Enflasyon Değişikliği:** Mevcut sistemde `purchaseCounts` sadece TEK oyuncunun alım sayısına göre. Online'da bu **TÜM oyuncuların** toplam alım sayısına göre olmalı. Böylece gerçek arz-talep enflasyonu oluşur.

3.4 USEBANKSTORE.JS → BANK SERVİCE + FOREX SERVİCE + IPO SERVİCE

Mevcut: ~700 satır, kredi + mevduat + FX + emtia vault + IPO.

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`activeLoan`, `loanInstallment`	**Server:** `LoanService` (DB)	`loans` tablosu
`applyForLoan()`	**Server:** `LoanService.apply()`	Kredi limiti sunucuda hesaplanır
`depositBalance`, `depositYield`	**Server:** `DepositService` (DB)	Mevduat sunucuda
`fxRates` (8 kur)	**Server:** `ForexService` (Redis)	Kurlar sunucuda simüle
`tickFX()`	**Server: Sovereign Pulse**	FX rate güncelleme
`forexPositions`	**Server:** `ForexPositionService` (DB)	`forex_positions` tablosu
`openForexPosition()`	**Server:** `ForexService.openPosition()`	Margin + likidasyon sunucuda
`checkForexLiquidations()`	**Server: Sovereign Pulse**	%90 margin loss → otomatik kapatma
`currencies` (EUR, TRY, vb.)	**Server:** `CurrencyService` (DB)	Kullanıcının döviz bakiyeleri
`commodities` (Altın, Gümüş)	**Server:** `VaultService` (DB)	Kullanıcının emtia bakiyeleri
`vaultRented`	**Server:** `VaultService`	Vault durumu DB'de
`ipoCompanies`	**Server:** `IPOService` (DB)	`ipo_listings` tablosu
`initiateIPO()`	**Server:** `IPOService.launch()`	%49 max kontrol sunucuda
`buyBackIPO()`	**Server:** `IPOService.buyback()`	Değerleme sunucuda
IPO değerleme formülü	**Server**	`calculateBusinessValuation()`
`tickIPOs()`	**Server: Sovereign Pulse**	IPO fiyat güncelleme

3.5 USEMEGACORPSTORE.JS → MEGACORP SERVİCE (HOLDİNG)

Mevcut: ~500 satır, hostile takeover + şube yönetimi + envanter + maaş.

MEVCUT ALAN/METHOD	NEREYE GİDECEK	DÖNÜŞÜM
`unlockedBoards`	**Server:** **`HoldingService`** (DB)	`holding_companies` tablosu
`megaBusinesses`	**Server:** **`HoldingService`** (DB)	Şube, envanter, personel DB'de
`activeMissions`	**Server:** **`MissionService`** (DB)	Misyon ilerlemesi DB'de
`unlockBoard()`	**Server:** **`HoldingService.unlock()`**	%75 hisse kontrolü sunucuda
`openBranch()`	**Server:** **`BranchService.construct()`**	30dk timer sunucuda
`hireStaff()`	**Server:** **`StaffService.hire()`**	$50K/10 kişi sunucuda
`restockInventory()`	**Server:** **`InventoryService.restock()`**	Maliyet sunucuda
`payWages()`	**Server:** **`WageService.pay()`**	Maaş borcu sunucuda
Envanter tüketimi (3 saat)	**Server: Sovereign Pulse**	`maxInventory / 1080` per pulse
Grev mekanizması (16 saat maaş borcu)	**Server: Sovereign Pulse**	İş durdurup gelir keser
Vergi mührü (tax seal)	**Server:** **`TaxService`**	Category freeze
`SECTOR_MAP` (4 sektör)	**Server** (config) + **Client** (constants)	`@bb/core/constants/sector-map`

3.6 USEENGİNESTORE.JS → TAMAMEN KALDIRILIR

Mevcut: ~600 satır, tüm oyun ekonomisinin kalbi.

MEVCUT FONKSİYON	YENİ KONUM	AÇIKLAMA
`startEngine()`	**Server:** **`SovereignPulseService`**	10s cron sunucuda
`performPulse()`	**Server:** **`SovereignPulseService.tick()`**	Tüm ekonomi güncelleme
`handleOfflineEarnings()`	**Server:** **`OfflineService.calculateDelta()`**	Reconnect'te sunucu hesaplar
`getGlobalNetWorth()`	**Server:** **`ForbesService.calculateNetWorth()`**	Net varlık sunucuda
`getTotalHourlyIncome()`	**Server:** **`IncomeService.getHourly()`**	Gelir hesabı sunucuda
`getStaticIncomeRatios()`	**Server:** **`IncomeService`**	Vergi oranları dahil

> **[BİLGİ / WARNING]:** **Bu en radikal değişikliktir.** `useEngineStore` mevcut oyunun "beyni"dir — tek bir `performPulse()` fonksiyonu ~200 satır kod ile tüm ekonomiyi yönetir. Online'da bu fonksiyon **sunucu tarafına** taşınacak ve her 10 saniyede bir tüm oyuncular için çalışacak.

3.7 USEFORBESSTORE.JS → FORBES SERVİCE

MEVCUT	YENİ	AÇIKLAMA
200 NPC milyarder (parodi)	**Server: Forbes seed data**	NPC'ler DB'de, gerçek oyuncularla karışık sıralama
`tickForbes()`	**Server:** `ForbesService.updateRankings()`	Redis Sorted Set ile O(log N) sıralama
`getPlayerRanking()`	**Server:** `ForbesService.getRank(userId)`	Redis ZREVRANK
NPC drift (volatilite)	**Server: Sovereign Pulse**	NPC net worth simülasyonu

Online Forbes Farkı:

```
MEVCUT: 200 NPC + 1 oyuncu → Sahte rekabet
YENİ:   200 NPC + 100,000+ gerçek oyuncu → Gerçek küresel rekabet
         └→ NPC'ler sadece ilk günlerde boşluk doldurmak için
             Gerçek oyuncu sayısı arttıkça NPC'ler irrelevant olur
```

3.8 USENOTİFİCATİONSTORE.JS → NOTİFİCATİON SERVİCE + PUSH

MEVCUT	YENİ	AÇIKLAMA
`showNotification()` (lokal toast)	**Client: toast + Server: push**	Uygulama açıkken toast, kapalıyken push
`toastLayerStack` (modal stacking)	**Client**	UI concern, aynen kalır

3.9 USEADSİMULATORSTORE.JS → AD VERİFİCATİON SERVİCE

MEVCUT	YENİ	AÇIKLAMA
`startAdSim()` (sahte reklam)	**Server:** `AdVerificationService`	Server-Side Verification (SSV) zorunlu
`onReward()` (direkt ödül)	**Server callback**	AdMob SSV → sunucu doğrular → ödül verir
VIP bypass (reklam atlama)	**Server: VIP check**	VIP durumu sunucudan kontrol

---

3.10 STORAGE.JS → TAMAMEN YENİDEN YAPILANDIRILIR

MEVCUT	YENİ	AÇIKLAMA
In-memory cache + MMKV disk	**PostgreSQL** (kaynak) + **MMKV** (sadece cache)	Veri kaynağı sunucu
`saveUser()`, `loadUser()` vb.	**REST API** calls	İlk yüklemede GET, sonra WS güncellemeler
5 saniyelik debounced write	**Kaldırılır**	Sunucu kendi persistence'ını yönetir
`prefetchStorage()`	**Kaldırılır**	WS bağlantısında state alınır

---

4. Ekran-by-Ekran Dönüşüm Planı

4.1 KAZANCLARSCREEN (ANA DASHBOARD / TIKLAMA)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
Kart tıklama → `handleManualClick()`	Client `TAP` WS event gönderir → Server hesaplar → `BALANCE_UPDATED` döner
Anti-cheat (stddev, interval analizi)	Kaldırılır → Server-side rate limiting (15 tap/s)
Overclock (reklam → 10x 30s)	Client reklam gösterir → AdMob SSV → Server `ACTIVATE_BOOSTER`
Federal Bailout (reklam → $50K)	Aynı SSV akışı
Floater animasyon (+$10 uçuş)	Client'ta kalır (UI concern)
Net varlık gösterimi	Server'dan gelen değeri gösterir
İşlem geçmişi	REST `GET /portfolio/history` + WS güncellemeler
Kart seviye ilerleme	Server'dan gelen `totalClicks / threshold`

Değişiklik Miktarı: %60 yeniden yazılır (iş mantığı kaldırılır, WS bridge eklenir)

4.2 YATİRİMSCREEN (BORSA TERMİNALİ)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
Hisse listesi (75 hisse)	REST ile ilk yükleme, sonra WS `STOCK_PRICES_UPDATED` ile delta update
Kripto listesi (20 token)	Aynı pattern
Fiyat sparkline (history[])	Server'dan son 60 veri noktası REST ile, sonra WS append
Alım/Satım	`BUY_STOCK` / `SELL_STOCK` WS command → `TRADE_CONFIRMED` / `TRADE_REJECTED`
Limit emir	`PLACE_LIMIT_ORDER` WS → `LIMIT_ORDER_FILLED` WS (asenkron)
Gayrimenkul	REST ile listing, `BUY_PROPERTY` WS command
Enflasyon (+10% per purchase)	Server'da GLOBAL alım sayısına göre hesaplanır
Filtreler (ülke, fiyat, temettü)	Client-side filter (veri zaten cache'de)
InstrumentDetailModal	Aynı UI, ama trade komutları WS üzerinden

Değişiklik Miktarı: %40 yeniden yazılır (veri kaynağı değişir, UI korunur)

4.3 ISLETMESCREEN (İŞ YÖNETİMİ)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
İşletme satın al	`BUY_BUSINESS` WS → Server bakiye + kapasite kontrol
Kapasite genişleme	`EXPAND_CAPACITY` WS → Server timer başlatır
İnşaat bypass (reklam)	AdMob SSV → Server `BYPASS_CONSTRUCTION`
Business Turbo (reklam → +20% 1s)	AdMob SSV → Server `ACTIVATE_BOOSTER`
Tasfiye (%80 refund)	`LIQUIDATE_BUSINESS` WS → Server hesaplar
TaxiBusinessScreen	Aynı pattern: araç alım/bakım/showroom WS üzerinden
LogisticsHub	Aynı pattern

Değişiklik Miktarı: %50 yeniden yazılır

4.4 BANKASCREEN (BANKA PORTALI)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
BankOverview (kredi + mevduat)	`APPLY_LOAN`, `REPAY_LOAN`, `DEPOSIT`, `WITHDRAW` WS commands
ExchangeTerminal (FX + CFD)	`EXCHANGE_CURRENCY`, `OPEN_FOREX`, `CLOSE_FOREX` WS commands
VaultManagement (emtia)	`TRADE_COMMODITY` WS command
IPOPortal	`LAUNCH_IPO`, `BUYBACK_IPO` WS commands
NewsBanner (market haberleri)	Server'dan WS `MARKET_EVENT` push
BankTicker	WS `FX_RATES_UPDATED` ile gerçek zamanlı

Değişiklik Miktarı: %50 yeniden yazılır

4.5 PROFİLSCREEN (PROFİL VE FORBES)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
Avatar upload	REST `PATCH /users/me/profile` (multipart) → S3/CDN
Forbes sıralama	WS `FORBES_UPDATED` + REST fallback
VIP Store	Client IAP → Server-side verification → bakiye güncelleme
Promo code	REST `POST /api/promo/redeem`
Vergi ofisi	`PAY_TAX`, `PAY_ALL_TAXES` WS commands
İstatistikler	REST `GET /users/me/stats`
DevPanel	**KALDIRILIR** → Server-side admin panel
Developer password	**KALDIRILIR** → RBAC admin role

Değişiklik Miktarı: %45 yeniden yazılır

4.6 BOARDSCREEN (MEGACORP HOLDİNG)

MEVCUT ÖZELLİK	DÖNÜŞÜM STRATEJİSİ
CEO toplu işlemler	`RESTOCK_ALL`, `PAY_ALL_WAGES` WS commands
Şirket detay	REST `GET /megacorp/:stockId`
Şube açma	`OPEN_BRANCH` WS → 30dk timer sunucuda
Personel alma	`HIRE_STAFF` WS → $50K/10 kişi sunucuda
Misyon	`START_MISSION`, `COMPLETE_MISSION` WS commands
Holding tasfiye	`LIQUIDATE_HOLDING` WS → %70 refund sunucuda
Hunt Radar	Server'dan portföy % hesabı

Değişiklik Miktarı: %55 yeniden yazılır

4.7 ONBOARDİNG / LOGİN EKRANLARI

MEVCUT	YENİ
`LanguageSelectScreen`	Aynen kalır + dil seçimi server'a kaydedilir
`OnboardingScreen` (username input)	→ `RegisterScreen` (email + password + username)
`ProfileSelectScreen`	Kaldırılır veya sadeleştirilir (login sonrası direkt game)

YENİ EKRANLAR EKLENİR:

- `LoginScreen` (email/password + Google + Apple)
- `RegisterScreen` (username + email + password)
- `ForgotPasswordScreen`

---

5. İş Kuralları Sunucu/Client Dağılım Tablosu

5.1 SUNUCUYA TAŞINACAK KURALLAR (SERVER-AUTHORİTATİVE)

#	İŞ KURALI	MEVCUT DOSYA	YENİ KONUM
1	Tıklama gücü = `clickPower × 2^(cardLevel-1) × (overclock ? 10 : 1)`	`useUserStore:252`	`ClickService`
2	Max click power: 750	`useUserStore:345`	`ClickService`
3	Anti-clicker: ≥20 click/1000ms → block, 3 strike → confiscate	`useUserStore:214-233`	`ClickService` (rate limit)
4	Kart milestone eşikleri: [100, 500, 2500, 10000, 50000]	`useUserStore:256`	`WalletService`
5	Kart bonusları: [$5K, $50K, $250K, $1M, $10M]	`useUserStore:256`	`WalletService`
6	Vergi kilidi: 16 saat ödenmezse kategori donar	`useUserStore:433`	`TaxService`
7	Vergi oranları: İşletme %20, G.Menkul %5-30, Hisse %15, Mevduat %10, Holding %20	`useEngineStore:31-110`	`TaxService`
8	Hisse fiyat simülasyonu (drift + mean reversion + noise)	`useMarketStore:280-340`	`PriceOracleService`
9	Kripto fiyat simülasyonu + flash pump/crash (%0.3 olasılık)	`useMarketStore:380-440`	`CryptoService`
10	Hisse tick clamp: [-8%, +8%]	`useMarketStore:328`	`PriceOracleService`
11	Kripto tick clamp: [-3.5%, +4.0%]	`useMarketStore:430`	`CryptoService`
12	Komisyon: %0.1 normal, %0.002 VIP	`useMarketStore:645`	`TradeService`
13	Kripto exposure limiti: $15M	`useMarketStore:636`	`TradeService`
14	Hostile takeover: %99 hisse → Board unlock	`useMarketStore:684`	`TakeoverService`
15	Hisse vergisi: %10 kâr üzerinden	`useMarketStore:737`	`TradeService`
16	G.Menkul enflasyonu: +10% per purchase count	`useBusinessStore:157`	`PropertyService`
17	G.Menkul stok limiti: 5/şablon	`useBusinessStore:150`	`PropertyService`
18	G.Menkul günlük değer artışı: %1/gün	`useBusinessStore:633`	`PropertyService`
19	G.Menkul günlük kira artışı: %0.5/gün	`useBusinessStore:640`	`PropertyService`

#	İŞ KURALI	MEVCUT DOSYA	YENİ KONUM
20	G.Menkul tasfiye: %85 net refund	`RealEstateDetailModal`	`PropertyService`
21	Araç kondisyon < 50% → gelir yarıya düşer	`useBusinessStore:61`	`VehicleService`
22	Araç emeklilik: kümülatif gelir ≥ maliyet × 1.20	`useBusinessStore:508`	`VehicleService`
23	Araç bakım maliyeti: `maintenanceCost × (wearTier / 100)`	`useBusinessStore:383`	`VehicleService`
24	İşletme kapasite genişleme: `(capacity/10) × $1M`, 60dk süre	`useBusinessStore`	`BusinessService`
25	İşletme tasfiye: %80 refund	`useBusinessStore`	`BusinessService`
26	Kredi faiz: %3.42	`useBankStore`	`LoanService`
27	Kredi max: Net varlığın %25'i	`BankOverview`	`LoanService`
28	Kredi vade: 24 saatlik taksit	`useBankStore`	`LoanService`
29	Mevduat getirisi: %2/saat	`useBankStore`	`DepositService`
30	FX kaldıraç: 1x, 10x, 25x, 50x, 100x	`ExchangeTerminal`	`ForexService`
31	Forex likidasyon: %90 margin kaybı	`useBankStore`	`ForexService`
32	Forex spread: `notional × 0.00005 × (1 + leverage/1000)`	`ExchangeTerminal`	`ForexService`
33	Vault kiralama: $5,000 (mevcut)	`useBankStore`	`VaultService`
34	IPO max satış: %49 (kontrol payı %51)	`useBankStore`	`IPOService`
35	IPO hisse sayısı: `max(1M, floor(valuation/25))`	`useBankStore`	`IPOService`
36	İş değerleme: `max(50K, capital + totalEarned×1.5 + hourly×80)`	`useBankStore`	`IPOService`
37	MegaCorp misyon: 5 aşama, maliyet `(step+1) × $1M`	`useMegaCorpStore`	`MissionService`
38	MegaCorp holding kapasite: $50M upgrade, +10 branch	`useMegaCorpStore`	`HoldingService`

#	İŞ KURALI	MEVCUT DOSYA	YENİ KONUM
39	Envanter tüketimi: `maxInventory / 1080` per pulse (3s exhaustion)	`useEngineStore`	`SovereignPulse`
40	Holding maaş: gross gelirin %35'i	`useEngineStore`	`WageService`
41	Holding maaş kilidi: 16s → grev	`useEngineStore`	`WageService`
42	Market cycle: 1-3 saat, %30 Bull/%30 Bear/%40 Neutral	`useMarketStore`	`EconomyEngine`
43	FX target drift: her 120 pulse (20dk)	`useBankStore`	`ForexService`
44	Safe haven temettü: 360 pulse'da (1s) Gold/Silver %0.05, CHF/EUR/GBP %0.03	`useBankStore`	`ForexService`
45	Offline cap: max 12 saat	`useEngineStore:540`	`OfflineService`
46	Promo kodları ve limitleri	`useUserStore`	`PromoService`
47	VIP günlük ödül: $50M/gün (UTC)	`useUserStore`	`RewardService`
48	Business turbo: 3600s, 1.20x gelir	`useUserStore`	`BoosterService`
49	Overclock: 30s, 10x tıklama	`useUserStore`	`BoosterService`
50	Forbes NPC volatilite: top10 0.00008, others 0.00015	`useForbesStore`	`ForbesService`

5.2 CLIENT'TA KALACAK KURALLAR (UI/PRESENTATİON ONLY)

#	KURAL	DOSYA	GEREKÇE
1	Floater animasyonu (+$10 uçuş yönü/süre)	`KazanclarScreen`	Tamamen görsel
2	Kart spring animasyonu (0.95 → 1.0 scale)	`KazanclarScreen`	Tamamen görsel
3	StaggeredCard giriş animasyonu (45ms delay)	`motion/StaggeredCard`	Tamamen görsel
4	Tab bar spring physics	`BottomTabNavigator`	Tamamen görsel
5	`formatMoney()` ($T, $B, $M, $K)	`useUserStore` → `@bb/core/utils`	Pure formatting
6	Sparkline SVG path hesaplama	`YatirimScreen`	Client-side rendering
7	Forbes Top 10 confetti	`ForbesScreen`	Tamamen görsel
8	Rank 1 EpicRankOneShow	`EpicRankOneShow`	Tamamen görsel
9	Timeframe seçici (15M, 1H, 4H, 8H)	`InstrumentDetailModal`	Veri filtresi (server'dan gelen history üzerinde)
10	Renk temaları (`theme.js`)	`theme.js`	Tamamen görsel
11	i18n dil seçimi ve çeviri	`i18n.js`, locales	Client-side
12	Cybernetic marquee animasyonu	`ProfileSelectScreen`	Tamamen görsel
13	Tutorial modal adımları	`TutorialModal`	Client-side flow

---

6. Migrasyon Risk Matrisi

RİSK	OLASILIK	ETKİ	AZALTMA STRATEJİSİ
Engine taşıma sırasında formül hatası	Yüksek	Kritik	Birim testler: mevcut formülleri TDD ile yeniden implemente et
WebSocket bağlantı kopmaları	Orta	Yüksek	Reconnection + offline catch-up + queue-based delivery
Race condition (aynı lotu 2 kişi alır)	Yüksek	Kritik	`SELECT ... FOR UPDATE` + `Serializable` isolation
Mevcut oyuncuların verisi kaybolur	Düşük	Kritik	Migrasyon scripti + backup + rollback planı
Performans sorunları (tick > 10s)	Orta	Yüksek	Profiling + BullMQ offloading + Redis cache
Apple App Store reject	Orta	Orta	Apple Sign-In zorunlu + IARC derecelendirme
Oyun dengesi bozulur (hiperenflasyon)	Orta	Yüksek	Otomatik stabilizatörler + admin kontrol paneli
DevPanel sızıntısı	Yüksek (mevcut)	Kritik	Tamamen kaldır + RBAC admin panel
DDoS saldırısı	Orta	Yüksek	Cloudflare + rate limiting + WAF
Veritabanı yavaşlama	Düşük	Yüksek	Index optimizasyonu + read replica + connection pool

---

7. Veri Dönüştürme Stratejisi (Mevcut Oyuncular)

7.1 OFFLİNE → ONLİNE GEÇİŞ SENARYOSU

```
Senaryo: Mevcut offline oyuncular verilerini kaybetmeden
online sisteme geçiyor.

Adımlar:
1. Oyuncu güncellemeyi indirir (yeni versiyon)
2. İlk açılışta "Online'a Geçiş" ekranı görür
3. Email/password ile hesap oluşturur
4. Client, MMKV'deki mevcut veriyi JSON olarak paketler
5. Server'a POST /api/migration/import gönderir
6. Server veriyi doğrular:
   ├── Balance: max $10T cap (makul limit)
   ├── Portfolio: mevcut hisse ID'leri geçerli mi?
   ├── Properties: hala müsait mi? (first-come-first-serve)
   ├── Anti-cheat: mantıksız değerler var mı?
   └── Vehicles: model ID'ler geçerli mi?
7. Doğrulanmış veri DB'ye yazılır
8. Oyuncu online sisteme dahil olur
```

> **[BİLGİ / CAUTION]:** **Migrasyon Riski:** Offline'da herkes sınırsız mülk/hisse alabilir. Online'da arz sınırlı. İlk gelenler mevcut varlıklarını korur, ama online'daki sınırlı arz havuzundan düşülür. Potansiyel çözüm: Migrasyon penceresi (ilk 7 gün) → sonra offline veri import kapatılır.

7.2 MMKV → POSTGRESQL VERİ EŞLEŞTİRMESİ

MMKV KEY	POSTGRESQL TABLO	NOTLAR
`@bb_user`	`users` + `wallets`	Balance, stats, card levels
`@bb_portfolio`	`stock_holdings`	Mevcut portföy pozisyonları
`@bb_stocks`	**Kullanılmaz**	Fiyatlar sunucu tarafında yeniden oluşturulur
`@bb_crypto`	**Kullanılmaz**	Fiyatlar sunucu tarafında
`@bb_businesses`	`businesses` + `vehicles`	İş + filo verileri
`@bb_real_estate`	`properties`	Mülk sahiplikleri
`@bb_limit_orders`	`limit_orders`	Açık emirler (fiyatlar değişmiş olabilir)
`@bb_bank_data`	`loans` + `forex_positions` + `currencies` + `commodities`	Banka verileri
`@bb_megacorp`	`holding_companies`	Holding ve şubeler
`@bb_forbes`	**Kullanılmaz**	Forbes sunucuda yeniden hesaplanır

---

8. Faz Faz Uygulama Planı

FAZ 0: HAZIRLIK (1-2 HAFTA)

```
├── Monorepo kurulumu (Turborepo + pnpm)
├── @business-bank/core paketi oluştur
├── @business-bank/socket-protocol paketi oluştur
├── Game constants'ları core'a taşı (kart seviyeleri, araç kataloğu, vb.)
├── docker-compose.yml (PG + Redis + NestJS)
├── Prisma schema oluştur (tüm tablolar)
├── Veritabanı seed data (75 hisse, 20 kripto, 20 mülk)
└── CI/CD pipeline (lint + test + build)
```

FAZ 1: AUTH + WALLET + CLİCK (4-6 HAFTA)

```
├── AuthModule: Register, Login, JWT, Google Sign-In
├── WalletModule: Balance CRUD, Tax tracking
├── ClickService: Tap processing, rate limiting, card levels
├── WebSocket Gateway: Connection, auth, basic rooms
├── Client adapters: useAuthStore, useWalletStore, useSocketStore
├── Mobile: Login/Register ekranları
├── TEST: 2 kullanıcı aynı anda tıklıyor, bakiyeleri ayrı
└── MILESTONE: Oyuncular giriş yapıp tıklayabiliyor
```

FAZ 2: STOCK EXCHANGE (4-6 HAFTA)

```
├── StockModule: Price oracle, tick simulation
├── TradeService: Buy/Sell with FOR UPDATE locking
├── OrderBookService: Limit orders
├── PortfolioService: Holdings tracking
├── DividendService: Hourly distribution
├── WebSocket: Price stream, trade confirmations
├── Client: YatirimScreen WS bridge
├── TEST: 100 kullanıcı aynı hisseyi alıyor, arz azalıyor
└── MILESTONE: Gerçek zamanlı borsa çalışıyor
```

FAZ 3: REAL ESTATE + FLEET (4-6 HAFTA)

```
├── PropertyModule: Limited supply, inflation, upgrades
├── AuctionService: Player-to-player property trading
├── BusinessModule: Create, upgrade, liquidate
├── VehicleModule: Buy, maintain, retire
├── GarageService: Capacity upgrades with timers
├── Client: IsletmeScreen + TaxiBusinessScreen WS bridge
├── TEST: Mülk stoku azalıyor, fiyatlar artıyor
└── MILESTONE: Gayrimenkul ve filo yönetimi online
```

FAZ 4: BANK + FOREX + IPO (3-4 HAFTA)

```
├── LoanModule: Apply, repay, installment scheduling
├── DepositModule: Yield calculation
├── ForexModule: Positions, leverage, liquidation
├── CommodityModule: Vault, trade
├── IPOModule: Launch, buyback, valuation
├── Client: BankaScreen WS bridge
└── MILESTONE: Tam bankacılık sistemi online
```

FAZ 5: MEGACORP + FORBES + ECONOMY ENGİNE (3-4 HAFTA)

```
├── MegaCorpModule: Holdings, branches, inventory, wages, strikes
├── MissionService: 5-step takeover missions
├── ForbesModule: Redis sorted set, NPC + player ranking
├── EconomyEngine: Full Sovereign Pulse (all modules integrated)
├── InflationService: M2 tracking, auto-stabilizers
├── Client: BoardScreen + ForbesScreen WS bridge
├── TEST: 1000 oyuncu aynı anda, ekonomi dengeleniyor
└── MILESTONE: Tam ekonomi simülasyonu çalışıyor
```

FAZ 6: CROSS-PLATFORM + POLİSH (4-6 HAFTA)

```
├── Web client (React + Vite): Tüm ekranlar
├── Responsive design (mobile + tablet + desktop)
├── Push notification (FCM + APNs + Web Push)
├── Offline/reconnection handling
├── Data migration tool (offline → online)
├── iOS build + Apple Sign-In
├── Admin panel (basic)
└── MILESTONE: Web + Android + iOS hepsi çalışıyor
```

FAZ 7: LAUNCH PREPARATİON (2-4 HAFTA)

```
├── Load testing (k6: 10K concurrent users)
├── Security audit
├── Performance optimization (query profiling, index tuning)
├── Monitoring dashboard (Grafana)
├── Alert rules setup
├── App Store / Play Store listing update
├── Privacy Policy + Terms update
├── Beta test (TestFlight + Play Console internal track)
└── MILESTONE: Soft launch → Public launch
```

---

9. Dönüşüm Sonrası Karşılaştırma

ÖZELLİK	MEVCUT (OFFLİNE)	YENİ (ONLİNE)
Oyuncu sayısı	1	Sınırsız
Veri saklama	Telefon hafızası (MMKV)	PostgreSQL + Redis
Hisse arzı	Sınırsız (herkes alabilir)	Sınırlı (1M lot paylaşılır)
Gayrimenkul	Sınırsız (5/şablon per player)	Sınırlı (5/şablon GLOBAL)
Fiyat mekanizması	Rastgele simülasyon	Gerçek arz-talep
Forbes	200 NPC vs 1 oyuncu	Gerçek oyuncular vs NPC'ler
Enflasyon	Yok (statik ekonomi)	Gerçek M2 bazlı enflasyon
Anti-cheat	Client-side (bypasslanabilir)	Server-authoritative
IAP doğrulama	Client-side (güvensiz)	Server-side verification
Güvenlik	Düşük (DevPanel, hardcoded password)	Yüksek (JWT, RBAC, rate limit)
Platform	Sadece Android	Android + iOS + Web
Rekabet	Sahte (NPC'lerle)	Gerçek (oyuncularla)
Ekonomik denge	Manuel (hardcoded)	Otomatik (stabilizatörler)
Veri kaybı riski	Yüksek (telefon silinirse)	Düşük (sunucu backup'ları)
Aylık maliyet	$0	$21-85 (başlangıç)

> **[BİLGİ / IMPORTANT]:** **Tüm raporların özeti:** 1. **Yazılım Mimari Raporu** → Nasıl yapılacağı (DDD, NestJS, monorepo) 2. **Teknik Altyapı Raporu** → Neyle yapılacağı (sunucu, DB, Redis, hosting) 3. **Platform Gereksinimleri** → Nerede çalışacağı (Android, iOS, Web) 4. **Bu Rapor (Uyum)** → Nereden nereye gidileceği (mevcut → yeni) Bu 4 rapor birlikte, projenin A'dan Z'ye tam bir yol haritasını oluşturmaktadır.

İŞBU TEKNİK ŞARTNAME MADDELERİ TETKİK EDİLEREK İMZA ALTINA ALINMIŞTIR

SİSTEM & OYUN BAŞ MİMARI

Erkam Şimşek

Erkam ŞİMŞEK

Yazılım Mühendisliği Direktörü

MÜHÜR & TASDİK

ŞİMŞEK GAME ★ ONAY

TEKNİK İCRA KURULU

Karar: 2026/084

ALTYAPI KOORDİNATÖRÜ

Sistem Heyeti

Bulut Sistemleri Grubu

Dağıtık Mimari Uzmanı